

PRISM: Precision Robustness in Spectral vs. Additive Modulation for Post-Training Quantization of LLM Adapters

Nicolás Andrey Orozco Giraldo*, Luis Eduardo Muñoz Guerrero*, Yony Fernando Ceballos†

*Universidad Tecnológica de Pereira, Pereira, Colombia
nicolas.oroazco@utp.edu.co, lemunozg@utp.edu.co

†Facultad de Ingeniería, Grupo de Ingeniería y Sociedad, Universidad de Antioquia, Medellín, Colombia
yony.cebaillos@udea.edu.co

Abstract—Not every parameter-efficient adapter can be fully folded into a model’s frozen weights before deployment. LoRA’s entire learned update lives in a low-rank matrix that merges directly into the weight it adapts. SVMO, one of three coupled operators in the S^3 adapter, modulates a weight matrix’s singular values and is the only one of the three that produces a mergeable weight delta at all; the other two, a continuous-depth flow and a cross-attention bridge, operate on activations at inference time and are never merged or quantized regardless of what happens to the backbone. On their own, without those other two operators, SVMO’s merge does almost nothing: on a held-out set where the frozen base model scores 7.05 perplexity, merging SVMO in moves it to 7.02, while merging LoRA moves it to 4.82. SVMO’s update is also $47\times$ smaller than LoRA’s in Frobenius norm. Both facts might suggest SVMO’s merge is the safer one to quantize: smaller change, smaller stakes. Measured post-training quantization says otherwise. At 4-bit symmetric per-channel quantization, the SVMO-merged model’s perplexity rises by 4.751 points and LoRA’s by 3.712, the opposite of what update size and task contribution alone would predict. A magnitude-matched control, rescaling LoRA’s update to exactly match SVMO’s Frobenius norm before quantizing, reproduces this ordering: at equal magnitude, LoRA disturbs the merged weights’ per-channel statistics less than SVMO does. For adapters like S^3 , where most of the learned behavior lives in components that never touch the frozen weights, the small remainder that does get merged can still be the more fragile part to quantize, regardless of how little it contributes to task performance or how much smaller its update is.

I. INTRODUCTION

Fine-tuned language models rarely run in the precision they were trained in. After training, the adapter is merged into the frozen base weights, and the resulting model is quantized for deployment: to 8-bit or 4-bit integers, for a GPU with less memory than training used, or for a CPU or phone with none of the accelerator support training assumed. This merge-then-quantize pipeline is standard practice, not a corner case. Formats built for exactly this path, GGUF’s Q4_K_M chief among them, expect a single merged, quantized file rather than a base checkpoint plus a separate adapter, which is why on-device runtimes such as `llama.cpp` load models this way by default. An adapter shipped for a phone, an air-gapped deployment, or any GPU too small to hold an unquantized

7B model in memory goes through this same merge-then-quantize step regardless of how it was trained. Whether a specific parameter-efficient fine-tuning (PEFT) method makes that second step easier or harder is rarely examined; the adapter’s job is usually judged to end at the accuracy it reaches during evaluation, before quantization enters the picture.

Not every adapter can be folded into the frozen weights the same way. LoRA’s [1] entire learned effect lives in a low-rank matrix BA that merges directly into W ; quantizing the merged model quantizes all of what LoRA learned. S^3 couples three operators, and only one of them touches W at all: SVMO modulates W ’s existing singular values through a bounded, saturating nonlinearity, rescaling a direction W already has by at most a fixed fraction and never introducing one it does not have. The other two, defined formally in Section III, act on activations at inference time rather than on W itself; neither produces a weight delta, and neither is touched by quantizing the backbone, merged or not.

On its own, without NMF and STB, SVMO’s merge does very little. On the held-out set used throughout this paper, it moves the frozen base model’s perplexity from 7.05 to 7.02; LoRA moves the same base model to 4.82. This asymmetry is not an artifact of this checkpoint: S^3 ’s own evaluation reports the same pattern, that SVMO without STB contributes almost nothing on its own [2], because most of the adapter’s benefit comes from the two operators that quantization can never touch anyway. The question this paper asks is narrower than comparing two adapters that do the same job. Of the two things that can actually be merged and quantized, LoRA’s full update and SVMO’s small remainder, does either one’s geometry make it more fragile under quantization, independent of how much each contributes to the model’s accuracy on its own?

A substantial literature studies quantization and LoRA together, but almost entirely in the opposite order: the base model is quantized first and LoRA is trained on top of it [3], quantization is folded into training itself [4], or the small adapter matrices are what gets quantized [5]. Merging a trained adapter into full-precision weights first and quantizing the merged result for deployment second is the ordinary path out-

side the quantization literature, and whether an adapter’s own geometry affects how well that merge survives quantization is the question Section II returns to.

It is tempting to answer that question by comparing the merged weights directly: whichever adapter changes W less should be the safer bet under quantization, since quantization error grows with the dynamic range and heavy-tailedness of a weight matrix’s entries. Applied to the real checkpoints above, this comparison has a clean answer: SVMO’s update is $47\times$ smaller in Frobenius norm, and it leaves the per-channel kurtosis of the merged weights essentially where it started, while LoRA’s shifts it by up to three orders of magnitude in the largest MLP projections. SVMO looks like the obviously safer choice, on top of already being the one that changes the model’s behavior less.

That comparison is confounded, and the confound is fixable. LoRA’s larger footprint could reflect something about how additive, unconstrained updates behave in general, or it could simply reflect that reaching a much larger change in the model’s output took a much larger change to the weights. Separating those two explanations only requires rescaling LoRA’s update, per matrix, to the same Frobenius norm as SVMO’s, and re-measuring. When we do this, the ranking reverses: at matched magnitude, LoRA’s update disturbs per-channel weight statistics less than SVMO’s does. The first comparison mistook update size for update safety.

The real test is whether this magnitude-controlled ranking predicts what happens under actual post-training quantization of the original checkpoints, before any artificial rescaling. It does. At 8-bit symmetric per-channel quantization, both adapters lose essentially nothing: the held-out perplexity change is within noise of zero for either. At 4-bit, where quantization error is large enough to matter, LoRA’s merged model degrades by 3.712 perplexity points and SVMO’s by 4.751, the opposite order from the naive comparison and the same order the magnitude-controlled experiment predicted in advance.

This paper reports that result and the two experiments behind it, in the order they were run: the naive comparison, the controlled experiment that overturns it, and the real quantization measurement that confirms the controlled experiment. Our contributions are:

- A demonstration that ranking PEFT adapters by the raw magnitude of their merged weight update, already common practice, can rank them backwards relative to how they behave under real post-training quantization.
- A magnitude-matched control that isolates the effect of an adapter’s update *geometry* (additive low-rank vs. bounded spectral rescaling) from its update *size*, and correctly predicts which adapter degrades less under real 4-bit quantization before that quantization is ever run.
- Measured perplexity change for both adapters at two quantization precisions, 8-bit and 4-bit, on real checkpoints of the same 7B base model: negligible for either adapter at 8-bit, and substantial and correctly ordered by the magnitude-controlled prediction at 4-bit.

II. RELATED WORK

A. Weight Outliers and Post-Training Quantization

Naive uniform quantization loses the most precision on the weights or activations that stray furthest from the bulk of the distribution. GPTQ [6] and its successors reduced this loss by calibrating per-channel scales against a small dataset, but even calibrated quantizers struggle when a distribution has a few entries orders of magnitude larger than the rest. AWQ [7] addresses this directly: it identifies the weight channels most sensitive to activation outliers and protects them with a channel-wise scaling factor before quantizing, rather than treating every channel identically. SpinQuant [8] approaches the same problem differently, learning a rotation matrix that redistributes a weight matrix’s energy away from extreme outliers before any rounding happens. Both methods exist because how spiky a matrix’s entries are, not only how large the matrix is, determines how much a low-bit quantizer damages it. The same concern leads GGUF’s Q4_K_M, the dominant 4-bit format for CPU and phone deployment via `llama.cpp` [9], to keep outlier-sensitive portions of a tensor at higher precision rather than quantizing every block uniformly. Our kurtosis and max-to-standard-deviation measurements in Section III track the same property; the question we ask is how a PEFT adapter’s merge changes that property, rather than how to correct for it afterward.

B. Quantization and Parameter-Efficient Fine-Tuning

The dominant way quantization and LoRA-style adapters meet in the literature runs in the opposite order from this paper. QLoRA [3] quantizes the frozen base model to 4-bit NormalFloat first and trains LoRA on top of the frozen, quantized weights, so the adapter learns to compensate for quantization noise that is already present during training. LoftQ [10] refines the initialization of that same pipeline: it alternates quantization and low-rank approximation to find a quantized base and an initial adapter that jointly minimize the Frobenius-norm gap to the original full-precision weight, before LoRA training even starts. QA-LoRA [11] allocates quantization in groups aligned with the adapter’s own structure, so the trained adapter can be folded back into the quantized base without a separate dequantization step. LowRA [5] moves quantization to the adapter side entirely, compressing the trained LoRA matrices A and B down to sub-2-bit precision. L4Q [4] folds quantization into the training loop itself.

Each of these methods answers a version of the question: how should we quantize, given that LoRA is also part of the picture. This paper asks a different question. Once an adapter has already been trained, merged back into full-precision weights, and judged on the accuracy that merge reaches, quantizing the result for deployment is a separate, later step, and the choice of adapter made earlier can still affect how much that later step costs. Train-then-merge-then-quantize is the ordinary path for any adapter evaluated first on accuracy and deployed only afterward; to our knowledge, no prior work compares adapters of different geometry along that specific path.

TABLE I
THIS PAPER’S QUESTION AGAINST THE CLOSEST PRIOR WORK.

Method	Order	What varies
QLoRA [3]	quantize, then train	training precision only
LoftQ [10]	quantize, then train	adapter initialization
QA-LoRA [11]	quantize, then train	quantization granularity
LowRA [5]	train, then quantize the adapter	adapter bit-width
L4Q [4]	quantize during training	training loop itself
This paper	train, merge, then quantize W	adapter geometry

Table I places this paper’s question against the closest prior work along the dimension that separates them: which side of quantization the adapter’s training happens on, and whether adapter geometry itself, rather than a specific quantization scheme, is what is being varied.

C. Spectral Adaptation

SVMO belongs to a small family of PEFT methods that adapt a frozen weight matrix through its singular value decomposition rather than through an additive term. Zhang and Pilanci’s Spectral Adapter [12] rotates or shifts the top singular vectors of W directly, with a parameter count that grows with the matrix’s rank. SVMO, introduced as part of the S^3 adapter [2], leaves U and V untouched instead and learns a bounded, pointwise nonlinear rescaling of the singular values Σ alone, through a small modulation network with $O(H^2)$ parameters independent of the matrix’s size. Values can only move by a fixed fraction of where they started. That bound is what motivates the naive expectation this paper tests, and overturns.

D. Other Points in the Additive Design Space

LoRA is not the only additive geometry an update can take, and the two adapters compared in this paper are one point each in a larger design space that this paper does not otherwise cover. DoRA [13] decomposes a pretrained weight into a magnitude vector and a direction matrix, then applies a LoRA-style low-rank update to the direction alone while the magnitude is trained separately; the merge still produces an unconstrained additive delta, but the delta is no longer isotropic across a row the way a plain BA product is. VeRA [14] goes the other way, freezing a single pair of shared random projection matrices across every adapted layer and training only a small per-layer scaling vector, which shrinks the trainable footprint far below LoRA’s at some cost in expressivity. Both remain additive rather than spectral, and both merge into W the same way LoRA does; whether their particular additive geometry disturbs post-quantization weight statistics the way plain LoRA’s does, more, or less, is a question this paper’s raw-versus-magnitude-matched method could answer but does not, since only one instantiation of each geometry, SVMO and LoRA $r = 8$, was actually measured.

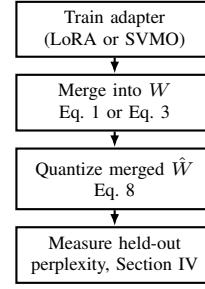


Fig. 1. The train-then-merge-then-quantize pipeline this paper measures. Both adapters pass through the same three steps; only the merge in step two differs by adapter geometry.

III. METHOD

A. The Two Mergeable Updates

Let $W \in \mathbb{R}^{d_{out} \times d_{in}}$ be a frozen weight matrix. LoRA [1] introduces $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$, trained so the adapted layer computes $Wx + \frac{\alpha}{r}BAx$. Merging folds this into the weight directly:

$$W_{\text{LoRA}} = W + \frac{\alpha}{r}BA, \quad (1)$$

with $r = 8$ and $\alpha = 16$ in the checkpoints used here.

SVMO instead operates on W ’s singular value decomposition, computed once offline: $W = U\Sigma V^\top$, truncated to the top k components $U_k \in \mathbb{R}^{d_{out} \times k}$, $\Sigma_k = \text{diag}(\sigma_1, \dots, \sigma_k)$, $V_k \in \mathbb{R}^{k \times d_{in}}$. A two-hidden-layer MLP $g_\theta: \mathbb{R} \rightarrow \mathbb{R}$ of width $H = 32$ learns a pointwise modulation of the log-singular values,

$$\Delta\sigma_i = \sigma_i \cdot \alpha \cdot \tanh(g_\theta(\log(\sigma_i + \varepsilon))), \quad i = 1, \dots, k, \quad (2)$$

with $\alpha = 0.3$ bounding the fractional change any singular value can undergo and $\varepsilon = 10^{-8}$. Merging gives

$$W_{\text{SVMO}} = W + U_k \text{diag}(\Delta\sigma) V_k^\top. \quad (3)$$

W_{LoRA} and W_{SVMO} have exactly the shape of W and are computed once, offline, before any quantization. Everything measured in Section IV operates on these merged matrices directly, not on W , A , B , or the SVD factors separately.

Fig. 1 summarizes the pipeline this paper measures end to end: an adapter is trained once, merged into W by whichever of Equations 1 or 3 applies, quantized by Equation 8, and evaluated on held-out perplexity. The only step that differs between the two adapters is the merge itself; training, quantization, and evaluation are identical procedures applied to whatever $\hat{W}$ each merge produces.

B. Why NMF and STB Stay Out of Scope

NMF defines a continuous-depth flow over a hidden state h , $dh/d\tau = f_\theta(h, \tau)$, integrated with RK4 at inference time; STB is a cross-attention module coupling h ’s projection onto U_k with the modulated singular values $\Sigma + \Delta\sigma$. Neither has the form $W \rightarrow W + \Delta$. Both require a forward pass through a small network at every token, at every layer, regardless of what precision the frozen backbone is stored in. Quantizing

W changes what W costs to store and multiply; it does not touch f_θ or STB’s coupling weights, which stay resident in full precision either way. That is the sense in which Section I calls SVMO the only one of the three operators quantization can reach at all.

C. Magnitude-Matched Control

For a given matrix, let Δ_{SVMO} and Δ_{LoRA} denote the two updates as trained, from Equations 3 and 1. To separate how much of LoRA’s larger footprint is geometry and how much is size, we construct a magnitude-matched counterfactual,

$$\Delta_{\text{LoRA}}^{\text{scaled}} = \frac{\|\Delta_{\text{SVMO}}\|_F}{\|\Delta_{\text{LoRA}}\|_F} \Delta_{\text{LoRA}}, \quad (4)$$

computed separately for each matrix, so that $\|\Delta_{\text{LoRA}}^{\text{scaled}}\|_F = \|\Delta_{\text{SVMO}}\|_F$ exactly.

Two properties of Equation 4 matter. First, the rescaling is isotropic: multiplying a matrix by a positive scalar changes its size, not its direction, so $\Delta_{\text{LoRA}}^{\text{scaled}}$ points exactly where the trained LoRA update points. Any change between the raw and the rescaled comparison is attributable to magnitude alone. Second, the rescaling is applied per matrix rather than once globally, because $\|\Delta_{\text{SVMO}}\|_F$ itself varies from one matrix to the next, set by that matrix’s own singular value spectrum; a single global scale would leave magnitude uncontrolled at the level Section IV actually measures.

Equation 4 produces a hypothesis, not a conclusion: if size rather than geometry explains why LoRA disturbs weight statistics more than SVMO in the raw comparison, that gap should shrink or reverse once magnitude is matched. What justifies treating the resulting prediction as more than a property of this one construction is that it is checked independently in Section IV: computed without touching a quantizer, it correctly anticipates which of the two *unscaled*, actually-trained checkpoints degrades less under real 4-bit quantization.

D. Weight-Distribution Metrics

For a merged matrix $W' \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, row i has entries $w_{i,1}, \dots, w_{i,d_{\text{in}}}$ with mean μ_i and standard deviation s_i . Two statistics are reported per matrix, each averaged over its d_{out} rows:

$$\kappa_i = \frac{1}{d_{\text{in}}} \sum_{j=1}^{d_{\text{in}}} \left(\frac{w_{i,j} - \mu_i}{s_i} \right)^4 - 3, \quad (5)$$

$$\rho_i = \max_j |w_{i,j}| / s_i, \quad (6)$$

excess kurtosis and the max-to-standard-deviation ratio. Both are zero and moderate, respectively, for a Gaussian row, and both grow when a few entries in the row sit far from the rest. They track the same property that per-channel quantizers such as GPTQ and AWQ calibrate against: a row with a small number of extreme entries needs a wider quantization range, which determines how coarsely every other entry in that row gets rounded.

The two statistics are deliberately redundant rather than complementary. κ_i weights every entry in the row by its

fourth power, so it is dominated by whichever entries are furthest from μ_i but still reflects the whole row’s shape; ρ_i looks at only the single largest entry, which is the one number that Equation 8 actually uses to set the quantization scale s_i for that row. Reporting both, rather than ρ_i alone, is a check against a spurious result specific to one statistic’s definition: if the raw-versus-magnitude-matched reversal in Section IV showed up in ρ_i but not κ_i , or the reverse, that would suggest the finding depended on the particular way one of the two statistics is computed rather than on anything about the adapters themselves. Tables III, IV, V, and VI show the same ordering under both, which is why the reversal is reported as a property of the update, not of the metric chosen to detect it.

E. Simulated Post-Training Quantization

Quantization is applied per output channel, matching the granularity of the metrics above. For b bits, with $q_{\text{max}} = 2^{b-1} - 1$, each row of a merged matrix is quantized and immediately dequantized:

$$s_i = \frac{\max_j |w_{i,j}|}{q_{\text{max}}}, \quad (7)$$

$$\hat{w}_{i,j} = s_i \cdot \text{clip}(\text{round}(w_{i,j}/s_i), -q_{\text{max}}-1, q_{\text{max}}). \quad (8)$$

$\hat{W}$ replaces W for every subsequent computation; the rounding happens once, offline, and all inference afterward runs in full precision on the dequantized values. This measures the effect of quantization error on model quality directly, without a low-bit compute kernel, and is standard for studying quantization sensitivity in isolation from quantization speed. We use $b = 8$ as a mild baseline and $b = 4$ as the level that matters for the deployment scenario motivating this paper, matching Q4_K_M’s bit-width (Section II) even though the block structure of Equation 8 is simpler.

Two simplifications separate Equation 8 from a deployed quantizer, and both push in the direction of understating rather than overstating quantization damage. First, GPTQ and AWQ calibrate their per-channel scales against a small set of representative activations, choosing s_i to minimize a reconstruction objective rather than reading it directly off $\max_j |w_{i,j}|$; a calibrated scale can absorb some of what a row’s single largest entry would otherwise force onto every other entry in that row. Second, Q4_K_M and similar deployed formats quantize in small blocks within a row rather than one scale per full row, which isolates an outlier to its own block instead of widening the range for the whole row. Equation 8 does neither: one scale, set by the single largest magnitude, for an entire row. Whatever gap in weight-distribution statistics or held-out perplexity Section IV reports is the effect of the plainest quantizer that still respects per-channel granularity; a calibrated, blocked quantizer would be expected to narrow the gap between adapters, not to reverse which one comes out ahead.

TABLE II
RAW COMPARISON, AVERAGED OVER 196 MATRICES.

	Base	+SVMO	+LoRA
Kurtosis κ	5.6745	5.6745	5.5853
Max/std ρ	4.8799	4.8799	4.8776
$\ \Delta\ _F/\ W\ $	—	0.051%	2.400%

IV. EXPERIMENTS

A. Setup

All checkpoints come from the same training run of Qwen2.5-7B-Instruct [15] (28 layers, hidden size 3584): SVMO with rank $k = 128$ and modulation width $H = 32$, and LoRA with $r = 8$ (Section III). Held-out perplexity is measured on the same sixteen examples used throughout the S^3 project, the last sixteen instruction-response pairs of the Alpaca training split, tokenized with the project’s standard template, 1,978 scored tokens in total. Every number below, including for the frozen base model with no adapter merged at all, comes from this same held-out set, streamed one decoder layer at a time so the full forward pass runs on a CPU with no GPU resident model.

Hardware and compute cost. The real quantization measurements reported later in Section IV ran on a single consumer CPU with no GPU resident model at any point, materializing one of the 28 decoder layers’ weights at a time from disk, running the held-out forward pass through it, and releasing it before the next layer loads; peak resident memory never holds more than one layer plus the token embedding and output head. Each of the four runs, base plus SVMO and LoRA at both bit-widths, took 62 to 64 minutes end to end. This is why the sample size and seed count in Section V-C are what they are rather than larger: a single held-out evaluation at this scale already costs roughly an hour on hardware with no accelerator, and a full retraining run to add a second seed to either adapter’s checkpoint is a separate, far larger cost that this paper’s hardware does not make casual to repeat.

B. The Raw Comparison

On this held-out set, the frozen base model scores 7.05 perplexity. Merging SVMO moves it to 7.02; merging LoRA moves it to 4.82. Across all $28 \times 7 = 196$ adapted matrices, SVMO’s update averages $47\times$ smaller than LoRA’s in Frobenius norm relative to $\|W\|$ (0.05% vs. 2.4%).

Table II reports the two weight-distribution statistics from Section III, averaged over all matrices. SVMO leaves both essentially where the frozen base model started; LoRA’s merge visibly shifts both.

The gap is not uniform across matrix types. Table III reports the mean absolute kurtosis shift by matrix; the largest MLP projections show the widest gap, with LoRA’s shift over three orders of magnitude larger than SVMO’s in `gate_proj`.

Kurtosis is not the only statistic that shows this pattern. Table IV reports the same by-matrix breakdown for the max-to-standard-deviation ratio ρ : the ordering matches Table III

TABLE III
MEAN $|\Delta\kappa|$ BY MATRIX, RAW COMPARISON.

Matrix	SVMO	LoRA	Ratio
q_proj	0.00013	0.00170	$13\times$
k_proj	0.00037	0.00264	$7\times$
v_proj	0.00025	0.00117	$5\times$
o_proj	0.00031	0.00121	$4\times$
gate_proj	0.00054	0.5996	$1104\times$
up_proj	0.00004	0.0152	$390\times$
down_proj	0.00020	0.00370	$19\times$

TABLE IV
MEAN $|\Delta\rho|$ BY MATRIX, RAW COMPARISON.

Matrix	SVMO	LoRA	Ratio
q_proj	0.00023	0.00108	$4.8\times$
k_proj	0.00039	0.00144	$3.7\times$
v_proj	0.00015	0.00116	$7.5\times$
o_proj	0.00035	0.00073	$2.1\times$
gate_proj	0.00034	0.00879	$25.9\times$
up_proj	0.00005	0.00533	$112.8\times$
down_proj	0.00010	0.00092	$9.2\times$

matrix for matrix, largest in `up_proj` and `gate_proj`, smallest in the attention projections, so the raw comparison’s conclusion does not depend on which of the two weight-distribution statistics from Section III is used to state it.

Taken at face value, this comparison says SVMO’s merge is the safer one to quantize: smaller in magnitude, and gentler on the statistics per-channel quantizers depend on.

C. Magnitude-Matched Control

Rescaling LoRA’s update per matrix to match SVMO’s Frobenius norm exactly (Equation 4) and repeating the same measurement reverses the ranking. At matched magnitude, LoRA disturbs kurtosis less than SVMO in 188 of 196 matrices, and the max-to-standard-deviation ratio less in 179 of 196. Table V repeats the by-matrix breakdown with LoRA’s update scaled down to SVMO’s size before measuring: the two projections that showed LoRA as over $1,000\times$ more disruptive in the raw comparison now show SVMO as the more disruptive one instead.

The reversal holds for ρ as well. Table VI repeats Table IV’s breakdown with LoRA’s update scaled to SVMO’s magnitude: every matrix that favored SVMO in the raw comparison now favors LoRA instead, the same reversal Table V shows for kurtosis.

D. Depth Dependence

Tables III and V average over all 28 layers, which hides how unevenly the raw gap is distributed by depth. Splitting the 196 matrices into an earlier half (layers 0–13) and a later half (layers 14–27), Table VII shows the mean absolute kurtosis shift in the raw comparison is $627\times$ larger for LoRA than SVMO in the earlier half, against $7.9\times$ in the later half: an order of magnitude difference in how lopsided the raw

TABLE V
MEAN $|\Delta\kappa|$ BY MATRIX, MAGNITUDE MATCHED.

Matrix	SVMO	LoRA (scaled)	Ratio
q_proj	0.00013	0.00001	0.05×
k_proj	0.00037	0.00004	0.10×
v_proj	0.00025	0.00000	0.02×
o_proj	0.00031	0.00000	0.01×
gate_proj	0.00054	0.00019	0.34×
up_proj	0.00004	0.00001	0.37×
down_proj	0.00020	0.00001	0.03×

TABLE VI
MEAN $|\Delta\rho|$ BY MATRIX, MAGNITUDE MATCHED.

Matrix	SVMO	LoRA (scaled)	Ratio
q_proj	0.00023	0.00001	0.03×
k_proj	0.00039	0.00005	0.13×
v_proj	0.00015	0.00002	0.13×
o_proj	0.00035	0.00001	0.03×
gate_proj	0.00034	0.00002	0.05×
up_proj	0.00005	0.00002	0.40×
down_proj	0.00010	0.00001	0.07×

comparison looks depending on where in the network it is measured.

Nearly all of that early-layer gap traces to a single matrix, `gate_proj`’s own kurtosis in its unmodified, frozen state is 709.6 at layer 0, 139.5 at layer 1, and 67.1 at layer 2, one to three orders of magnitude larger than every other matrix at those depths and larger than `gate_proj` itself becomes at any later layer. This is the same weight-outlier concentration that motivates AWQ, SpinQuant, and GPTQ’s calibrated quantizers (Section II): a small number of channels, concentrated in specific early layers, that already sit far outside the bulk of the distribution before any adapter touches them. Measured against that already-extreme baseline, LoRA’s unconstrained update produces a larger absolute kurtosis shift than SVMO’s bounded one, the same direction as everywhere else in Table III, just amplified by the size of what it is perturbing.

The open question is whether the magnitude-matched reversal from Section III-C survives at these same extreme layers, or is itself an artifact of the calmer, mid-network matrices where most of the 196 comparisons live. It survives: among the 21 matrices making up layers 0 through 2, magnitude-matched LoRA disturbs kurtosis less than SVMO in 20, the one exception being `up_proj` at layer 2, by a margin of 0.00003. The reversal this paper reports is not a property of typical layers that happens to break down where outliers are largest; it holds at the one part of the network where the raw comparison is most lopsided in the opposite direction.

E. Real Post-Training Quantization

Table VIII reports measured perplexity for both adapters, at both quantization levels, on the original, unscaled checkpoints.

At 8-bit, neither adapter’s merge is disturbed enough to matter: both changes are near zero (0.002 to 0.021 perplexity

TABLE VII
MEAN $|\Delta\kappa|$, RAW COMPARISON, SPLIT BY DEPTH.

	SVMO	LoRA	Ratio
Layers 0–13 (early)	0.00028	0.17671	627×
Layers 14–27 (late)	0.00024	0.00193	7.9×

TABLE VIII
MEASURED PERPLEXITY, ORIGINAL CHECKPOINTS.

	fp32	8-bit	4-bit
Base (no adapter)	7.05	—	—
+SVMO	7.02	7.04 (+0.021)	11.77 (+4.751)
+LoRA	4.82	4.82 (+0.002)	8.53 (+3.712)

points), a small fraction of the base model’s own 7.05. At 4-bit, the gap is large, and it confirms the magnitude-matched control’s prediction: SVMO’s merged model degrades more (+4.751) than LoRA’s (+3.712), despite SVMO’s update being 47× smaller and despite SVMO contributing a fraction of LoRA’s improvement over the frozen base model to begin with.

V. DISCUSSION

A. Why Geometry Beats Magnitude

SVMO’s update is small in aggregate norm but not small in a way that spreads evenly across a row. It reweights whichever singular directions W already has, by a bounded multiplicative factor α , and pretrained weight matrices concentrate a large share of their row-wise magnitude in a handful of dominant directions to begin with. Rescaling those directions, even by a small factor, moves entries that were already near a row’s maximum, and a row’s maximum magnitude is exactly what sets s_i in Equation 8: the quantization scale for that row. LoRA’s update, by contrast, is an unconstrained low-rank addition; nothing ties its direction to W ’s existing spectrum, so at matched magnitude it has no particular reason to land on the entries a row’s own scale depends on, and Table V suggests it mostly does not. Update size predicts how much a merge changes the model. It does not predict, on its own, which entries within a row absorb that change, and post-training quantization only cares about the second question.

B. Implications for Adapter Design

For a practitioner choosing between candidate adapters ahead of a merge-then-quantize deployment, this separates two questions that a norm-based comparison collapses into one. Whether an adapter changes a model’s behavior more or less is answered by its Frobenius norm, or better, by held-out accuracy directly. Whether that change survives quantization is a different question, answered by where the update lands relative to the frozen weight’s own structure, not by how large the update is. The magnitude-matched control from Section III-C is one way to check the second question before a quantizer ever runs; the checkpoints measured here confirm it predicts the real measurement’s ordering, though one confirmation on

one model and one adapter pair does not make the control validated in general. Extending it to other spectral and additive methods is the natural next test.

C. Limitations

- **Single model, single run.** All numbers come from one training run of Qwen2.5-7B-Instruct per adapter, with no repeated seeds; the perplexity deltas in Section IV are one measurement each, not a distribution, and their variance is unknown.
- **Two adapters, one instance each.** The geometric contrast this paper draws, bounded spectral rescaling against unconstrained low-rank addition, comes from exactly one instantiation of each: SVMO at $k = 128$ and LoRA at $r = 8$. Other spectral or additive PEFT methods, DoRA, VeRA, the Spectral Adapter of Section II, were not measured, so the argument above interprets these two checkpoints rather than proving a claim across the design space.
- **Simulated, not deployed, quantization.** Equation 8 rounds and immediately dequantizes; it isolates quantization error from quantization speed, but a real GPTQ or AWQ deployment calibrates its scales against activations as well as weights, and may not degrade in exactly the proportions measured here.
- **A small held-out set.** Sixteen examples and 1,978 scored tokens are enough to establish which adapter degrades more under 4-bit quantization and by roughly how much, not enough to bound that gap tightly or rule out that a larger evaluation set would narrow it.
- **Two bit-widths.** We measured 8-bit and 4-bit only; where between them the gap between the two adapters starts to open was not localized.

VI. CONCLUSION

This paper asked a narrower question than which adapter reaches higher accuracy: once training ends and an adapter’s update is merged into full-precision weights, which merge survives the next step, quantization for deployment, better? For S^3 , only one of three coupled operators, SVMO, produces a mergeable weight delta at all; LoRA’s entire learned effect does. Comparing those two mergeable updates directly, rather than the adapters’ downstream accuracy, is what Section IV measures.

The comparison one would reach for first, ranking adapters by how much they change the merged weights, gets the answer backwards here. SVMO’s update is smaller by every measure available before quantization: $47\times$ smaller in Frobenius norm, and close to inert on the per-channel statistics quantizers depend on. Under real 4-bit quantization it is the one that degrades more, by 4.751 perplexity points against LoRA’s 3.712. A magnitude-matched control, rescaling LoRA’s update to SVMO’s own size before comparing, predicts this reversal before any quantizer runs: at equal magnitude, LoRA disturbs per-channel statistics less than SVMO does, in most of the 196 matrices checked. Update size and update safety are not the

same property, and a comparison built on the first can point the wrong way on the second.

The point extends past this one pair of adapters. Any parameter-efficient method judged on the accuracy it reaches, then merged and quantized only afterward for deployment, carries an unstated assumption that its merge behaves the way its size suggests. That assumption held for neither adapter measured here, in the direction expected. Checking it directly, with a magnitude-matched control or a real quantization pass, costs little next to the training already spent producing the adapter in the first place. Extending that check to other spectral and additive PEFT methods, to calibrated quantizers such as GPTQ or AWQ rather than the simulated round-trip used here, and to bit-widths between 8 and 4, would show whether the ordering found on this one model and this one adapter pair holds more generally or is specific to the checkpoints measured.

Code for every measurement reported in this paper, including the streaming implementation that makes CPU-only evaluation of a 7B model tractable, is available at [16].

REFERENCES

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [2] N. A. Orozco Giraldo, L. E. Muñoz Guerrero, and Y. F. Ceballos, “ S^3 : Spectral-spatial-smooth fine-tuning for large language models on 4gb consumer gpus,” 2026, manuscript submitted for publication.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] H. Jeon, Y. Kim, and J.-J. Kim, “L4q: Parameter efficient quantization-aware fine-tuning on large language models,” *arXiv preprint arXiv:2402.04902*, 2024, published at ACL 2025.
- [5] Z. Zhou, Q. Zhang, H. Kumbong, and K. Olukotun, “Lowra: Accurate and efficient lora fine-tuning of llms under 2 bits,” *arXiv preprint arXiv:2502.08141*, 2025, published at ICML 2025.
- [6] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [7] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, 2024, pp. 87–100.
- [8] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, “Spinquant: Llm quantization with learned rotations,” *arXiv preprint arXiv:2405.16406*, 2024.
- [9] G. Gerganov, “llama.cpp: Llm inference in c/c++,” 2023, gitHub repository, <https://github.com/ggml-org/llama.cpp>.
- [10] Y. Li, Y. Yu, C. Liang, P. He, N. Karampatziakis, W. Chen, and T. Zhao, “Loftq: Lora-fine-tuning-aware quantization for large language models,” *arXiv preprint arXiv:2310.08659*, 2023, published at ICLR 2024.
- [11] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian, “Qa-lora: Quantization-aware low-rank adaptation of large language models,” *arXiv preprint arXiv:2309.14717*, 2023, published at ICLR 2024.
- [12] F. Zhang and M. Pilanci, “Spectral adapter: Fine-tuning in spectral space,” *arXiv preprint arXiv:2405.13952*, 2024.
- [13] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, “Dora: Weight-decomposed low-rank adaptation,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [14] D. J. Kopiczko, T. Blankevoort, and Y. M. Asano, “Vera: Vector-based random matrix adaptation,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

- [15] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [16] N. A. Orozco Giraldo, L. E. Muñoz Guerrero, and Y. F. Ceballos, “PRISM: Precision robustness in spectral vs. additive modulation,” 2026, gitHub repository, <https://github.com/Anndy-bit/prism-spectral-quantization>.